

CICD规范

技术规范文件

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

目录

- 1. 概述
 - 1.1 目的
 - 1.2 适用范围
- 2. 分支策略
 - 2.1 推荐模式：Trunk-Based Development（主幹开发）
 - 2.2 分支命名规范
 - 2.3 分支保护规则
- 3. 构建流水线
 - 3.1 流水线阶段
 - 3.2 各阶段要求
 - 3.3 CI 脚本规范
- 4. 制品管理
 - 4.1 制品类型
 - 4.2 镜像/包命名规范
 - 4.3 版本号规范
- 5. 环境 Promotion
 - 5.1 环境定义
 - 5.2 Promotion 流程
 - 5.3 Promotion 审批矩阵
- 6. 质量门禁
 - 6.1 门禁规则
 - 6.2 门禁豁免
- 7. 附录
 - 7.1 参考文档
 - 7.2 工具推荐

CICD 规范

> 参考：Google 持续交付实践、ThoughtWorks 技术雷达、阿里云云效 CICD 最佳实践

1. 概述

1.1 目的

规范持续集成和持续交付流程，确保代码从提交到上线的全过程自动化、可追溯、高质量。

1.2 适用范围

适用于技术开发生所有自研系统的代码仓库管理、构建流水线、制品管理和环境 promotion 流程。

2. 分支策略

2.1 推荐模式：Trunk-Based Development（主幹开发）

对于大多数保险核心系统团队（5-20 人），推荐 Trunk-Based Development：

```
main
├── feature/xxx 开发新功能 < 300
│   └── → main
├── fix/xxx 修复问题
│   └── → main
└── release/v{major}.{minor} 发布版本
    └── → main + Tag
```

2.2 分支命名规范

分支类型	命名规范	示例
主幹	main 或 master	main
功能分支	feature/{JIRA-ID}-{}	feature/PROJ-123-add-channel-code
修复分支	fix/{JIRA-ID}-{}	fix/PROJ-456-fix-npe
发布分支	release/v{major}.{minor}	release/v2.3
热修复	hotfix/{JIRA-ID}-{}	hotfix/PROJ-789-critical-fix

2.3 分支保护规则

- main 分支禁止直接推送（需 PR + CI 通过 + Code Review）
- PR 必须至少 1 人 Approve 后方可合并
- 合并前必须通过所有 CI 流水线
- 合并方式推荐：Squash Merge（保持主幹历史清晰）

3. 构建流水线

3.1 流水线阶段

```
# .gitlab-ci.yml / GitHub Actions / Jenkinsfile
stages:
- lint # 代码检查
- build # 构建
- unit-test # 单元测试
- code-scan # 代码扫描
- package # 打包 Docker / Jar / 其他
```

- integration # 0000
- artifact # 0000
- deploy-staging # 000000
- e2e-test # 000000

3.2 各阶段要求

阶段	触发条件	超时	质量门禁
lint	Push/PR	5 min	无 Error
build	Push/PR	10 min	编译成功
unit-test	Push/PR	10 min	覆盖率 > 80%, 全部通过
code-scan	Push/PR	15 min	无 P0/P1 漏洞
package	Push to main / Tag	10 min	构建成功
integration	Push to main	20 min	全部通过
e2e-test	Pre-release	30 min	核心场景通过
deploy-staging	Tag / manual	10 min	健康检查通过

3.3 CI 脚本规范

```
# 0000GitHub Actions
name: CI Pipeline
on:
  push:
    branches: [main, release/**]
  pull_request:
    branches: [main]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run linter
        run: ./gradlew checkstyleMain
  build:
    runs-on: ubuntu-latest
    needs: [lint]
    steps:
      - uses: actions/checkout@v4
      - name: Build
        run: ./gradlew build -x test
      - name: Save artifact
        uses: actions/upload-artifact@v3
        with:
          name: app-jar
          path: build/libs/*.jar
  unit-test:
    runs-on: ubuntu-latest
    needs: [build]
    steps:
      - uses: actions/checkout@v4
      - name: Unit tests
        run: ./gradlew test
      - name: Coverage report
        uses: actions/upload-artifact@v3
        with:
          name: coverage-report
```

path: build/reports/jacoco/

4. 制品管理

4.1 制品类型

制品类型	格式	存储
Java 应用	Jar / War	Nexus / Artifactory
前端应用	静态文件 / Docker 镜像	Harbor / Docker Hub
Python 包	Wheel	PyPI 私有源
Docker 镜像	OCI 镜像	Harbor
Helm Chart	Chart 包	Harbor

4.2 镜像/包命名规范

```
# Docker 镜像
registry.company.com/{team}/{service-name}:{version}
# 示例
registry.company.com/insurance/policy-service:v2.3.0
registry.company.com/insurance/underwriting-api:v1.5.2
# Jar 包
com.company.insurance.{service}-{version}.jar
# 示例
com.company.insurance.policy-service-2.3.0.jar
```

4.3 版本号规范

遵循 Semantic Versioning 2.0 :

```
{major}.{minor}.{patch}[-{pre-release}][+{build}]
示例
v2.3.0 # 生产版本
v2.3.0-rc.1 # 预发布版本
v2.3.0-alpha.1 # 预发布版本
v2.3.1 # 生产版本
```

版本位	递增条件
major	不兼容的 API 变更、重构、重大功能
minor	向下兼容的新功能、非关键接口变更
patch	向下兼容的 Bug 修复

5. 环境 Promotion

5.1 环境定义

环境	用途	部署方式	数据
dev	开发自测	开发人员手动	模拟数据
test	功能测试	CI 自动部署	脱敏数据子集
staging	预发布验证	CI 自动部署	脱敏数据（接近生产）
production	正式环境	手动审批 + 自动部署	真实数据

5.2 Promotion 流程

```
dev > test > staging > production
□□□
- □□□□ main □□□ release □□ promotion
- □□□□ promotion □□□□□□□□□□□□
- staging → production □□□□□
- □□□□□□□ staging□□□□□
```

5.3 Promotion 审批矩阵

Promotion	自动	审批人	备注
dev → test	是 (CI 通过后)	无需审批	自动触发
test → staging	是	技术负责人	Release Tag 触发
staging → prod	否	技术负责人 + QA + 架构师	上线审批
hotfix → prod	是 (紧急模式)	紧急变更审批组	事后补审批

6. 质量门禁

6.1 门禁规则

门禁	阻断条件	处理方式
编译	编译失败	阻塞合并，通知提交者
单元测试	失败 / 覆盖率低于阈值	阻塞合并
代码扫描	P0/P1 漏洞	阻塞合并
集成测试	核心用例失败	阻塞 promotion
安全扫描	高危漏洞	阻塞 promotion
依赖扫描	已知 CVE	阻塞 promotion (根据严重级别)

6.2 门禁豁免

- P2 以下问题可由技术负责人批准豁免
- 安全高危漏洞不可豁免，必须修复
- 紧急修复可临时关闭特定门禁，但需在 24h 内补修

7. 附录

7.1 参考文档

- Google DevOps 持续交付
- GitLab CI/CD 官方文档
- Semantic Versioning 2.0
- 《发布变更管理规范》

7.2 工具推荐

用途	推荐工具	备选
代码仓库	GitLab / GitHub	Bitbucket

用途	推荐工具	备选
CI 引擎	GitHub Actions / GitLab CI	Jenkins
制品仓库	Nexus / Artifactory	Harbor
容器镜像仓库	Harbor	Docker Registry
配置中心	Nacos / Apollo	Consul
部署编排	Kubernetes + Helm	Docker Compose
IaC	Terraform	Pulumi / Ansible